

Plant Disease Prediction Using Symptoms Analysis

Mo. Furqan Qureshi
IT Department
Acropolis Institute of Technology &
Research
furqanqureshi231005@acropolis.in

Sudarshan Rajput
IT Department
Acropolis Institute of Technology &
Research
sudarshanrajput230850@acropolis.in

Prof. Mahendra K Verma
IT Department
Acropolis Institute of Technology &
Research
mahendraverma@acropolis.in

Yogesh Mali
IT Department
Acropolis Institute of Technology &
Research
yogeshmali230064@acropolis.in

Zoya Qureshi
IT Department
Acropolis Institute of Technology &
Research
zoyaqureshi231171@acropolis.in

Prof. Pawan K Gupta
IT Department
Acropolis Institute of Technology &
Research
pawangupta@acropolis.in

Abstract— *Plant diseases cause annual crop losses of twenty to forty percent globally, severely threatening agricultural productivity and farmer livelihoods, particularly in developing nations. Existing AI-based detection systems rely predominantly on image classification, which requires high-resolution photographs and stable internet connectivity — conditions rarely available in rural farming environments. This paper proposes a symptom-based Plant Disease Prediction System that allows farmers to describe observable plant conditions through a simple web interface, eliminating image dependency. A Random Forest classifier trained on 4,460 symptom records covering fifteen diseases across five crop categories achieved a prediction accuracy of 93.4%, outperforming Naive Bayes and Logistic Regression. The system delivers predictions in an average of 1.2 seconds through a lightweight Flask-based web application, providing disease identification, confidence scores, and treatment recommendations accessible to non-technical users..*

Keywords— *Plant Disease Prediction, Artificial Intelligence, Machine Learning, Smart Agriculture, Symptom Analysis, Random Forest, Crop Health Monitoring, Flask Framework, Deep Learning*

I. INTRODUCTION

Agriculture has always been the backbone of human civilization. Even today, it remains one of the most vital sectors supporting global food security and economic stability. The Food and Agriculture Organization (FAO) estimates that plant diseases alone are responsible for twenty to forty percent of global crop losses every year — a staggering figure that directly impacts the livelihoods of millions of farming families, particularly in countries like India where a large portion of the population depends on agriculture for survival.

The challenge of plant disease management begins not with treatment, but with identification. In practice, this is where the system most frequently fails. Farmers, particularly those operating small holdings in rural areas, often lack the knowledge required to distinguish between different disease conditions based on visual observation alone. By the time a disease is recognized and expert consultation is sought, it may have already spread across much of the crop, making early intervention impossible and causing losses that could have been avoided.

The growing availability of Artificial Intelligence and Machine Learning technologies has opened promising new avenues for automating disease detection. Deep learning models, particularly Convolutional Neural Networks, have shown impressive accuracy in identifying plant diseases from leaf images. However, these systems come with a set of practical requirements — high-resolution photographs, stable internet connectivity, and basic digital literacy — that much of the rural farming population simply cannot meet consistently. A tool designed only for ideal conditions is, in practical terms, a tool designed for the wrong audience.

This research presents a Plant Disease Prediction System built around symptom analysis rather than image classification. Farmers can describe what they observe — yellowing leaves, dark spots, wilting, fungal growth, leaf curling — through a straightforward web interface. The system processes these descriptions using machine learning algorithms trained on real plant disease data and returns a predicted disease, a confidence score, and actionable treatment guidance. The application is built using Python, Flask, Scikit-learn, HTML, CSS, JavaScript, MySQL, and Firebase, keeping it lightweight and deployable even in resource-limited environments. The goal is not to replace agricultural expertise but to put a reliable first line of defense in the hands of every farmer, regardless of where they are or what equipment they have access to.

II. LITERATURE REVIEW

Research on AI-based plant disease detection has grown considerably over the past decade, with scientists exploring everything from basic image processing to sophisticated deep learning architectures. What follows is a review of the most relevant work in this space and an honest assessment of where the gaps remain.

Mohanty, Hughes, and Salathé (2016) produced one of the most cited studies in this field, training a Convolutional Neural Network on the PlantVillage dataset — a collection of over 54,000 labeled leaf images spanning 26 crop species and 38 disease categories. Their model achieved 99.35% accuracy under laboratory conditions, which was a remarkable result at the time. The problem, as the authors themselves acknowledged, was that performance dropped sharply when the same model was tested on images collected in actual fields. Variations in lighting, background, and image angle introduced noise the model had never seen, revealing a generalization problem that continues to affect image-based systems to this day.

Too et al. (2019) built on this work by comparing several fine-tuned deep learning architectures — VGG, Inception, and ResNet — on plant disease classification tasks. They demonstrated that transfer learning, the practice of starting with a model pre-trained on general image data, meaningfully improves performance even when domain-specific training data is limited. The results were encouraging, but the underlying dependency on high-quality photographs remained unchanged.

Liakos et al. (2018) took a broader view, publishing a comprehensive review of machine learning applications across agriculture — covering crop yield prediction, soil management, weed detection, and disease identification. One of their more practically significant observations was that classical algorithms like Random Forest, Support Vector Machines, and Naive Bayes remain highly competitive in agricultural settings where interpretability and computational efficiency matter more than marginal accuracy gains. This finding was an important reference point in our decision to evaluate classical models rather than defaulting to deep learning.

Ferentinos (2018) evaluated several well-known deep learning architectures including AlexNet and GoogLeNet specifically for plant disease detection, reporting strong results across multiple crops. The study was technically thorough but also underscored the significant computational resources required to train and run such models — a practical barrier that is rarely discussed with sufficient honesty in the literature.

The most directly relevant prior work is that of Sharma, Patel, and Verma (2021), who proposed a symptom-based prediction model using Naive Bayes that achieved around 84% accuracy without relying on any image data. This was a meaningful contribution because it demonstrated that useful disease prediction is possible through structured symptom input alone. However, their system was narrow in scope, covering only a limited number of diseases, and was never developed into a deployable application with a proper interface, confidence scores, or treatment output. Our work picks up where theirs left off.

More recently, Sharma et al. (2023) explored hybrid CNN architectures designed to handle imbalanced datasets more effectively, and Chen, Wang, and Zhao (2024) made a case for integrating Explainable AI techniques into disease detection systems — arguing, convincingly, that farmers are more likely to trust and act on predictions they can understand.

Looking across this body of work, a clear pattern emerges. Most systems are built with the implicit assumption that users have reliable internet, capable smartphones, and the technical confidence to capture and submit quality images. For research settings, this assumption is reasonable. For rural India, it is not. The proposed system is designed with that reality in mind.

TABLE I: Summary of Related Research Studies

Year	Author(s)	Method	Key Contribution	Limitation
2016	Mohanty et al.	CNN (PlantVillage)	99.35% accuracy	Poor real-world generalization
2018	Liakos et al.	ML Review (RF, SVM, NB)	Validated classical ML	No specific prediction model
2018	Ferentinos	AlexNet, GoogLeNet	High multi-crop accuracy	High computational requirement
2019	Too et al.	Transfer Learning	Improved accuracy with limited data	Image quality dependency
2021	Sharma et al.	Naive Bayes (Symptom)	Symptom prediction without images	No UI, limited disease scope

2023	Sharma et al.	Hybrid CNN	Reduced overfitting	Complex architecture
2024	Chen et al.	Explainable AI	Improved transparency	Limited real-world deployment

III. RELATED WORK

Several commercial applications and research-based tools have emerged in recent years that attempt to solve the plant disease detection problem. Understanding what these systems do well — and where they fall short — is essential context for appreciating what the proposed system contributes.

Plantix, developed by PEAT GmbH in Berlin, is arguably the most widely used plant disease detection app available today. It uses deep learning image classification to identify diseases, nutrient deficiencies, and pest infestations across more than 60 crops and supports multiple languages. On paper, it is impressive. In practice, its usefulness depends entirely on the quality of the photograph submitted. Poor lighting, a shaky hand, or an aging smartphone camera can all compromise the prediction. More fundamentally, a farmer who cannot capture a usable image receives no assistance at all. For the rural user the app is ostensibly designed to help, this is a significant limitation.

Leaf Doctor, developed at the University of Hawaii, takes a narrower approach. Rather than identifying a specific disease, it estimates what percentage of a leaf's surface area has been affected using color segmentation. This can be useful for tracking disease progression over time, but it tells a farmer nothing about what is actually wrong with their plant or what they should do about it. It is a measurement tool, not a diagnostic one, and it still requires a clear, well-lit photograph to function.

AgroAI positions itself as a comprehensive smart farming platform, combining crop health monitoring, weather data, and soil condition analysis in a single interface. It is the most feature-rich of the three systems reviewed here, but that breadth comes at a cost. The interface is complex, the disease prediction module still relies on image input, and the overall experience assumes a level of technical comfort that many rural farmers do not have.

Beyond these commercial products, numerous research prototypes have been published — CNN-based classifiers trained on PlantVillage data, transfer learning models built on VGG16 and ResNet50, ensemble methods combining multiple classifiers. These systems

consistently demonstrate strong accuracy in controlled experiments and just as consistently fail to make the leap into real-world deployment. The reasons are predictable: high computational requirements, no accessible user interface, and a fundamental dependency on high-quality training images that mirrors the same limitations found in commercial tools.

What none of these systems offer is a practical middle ground — something between a manual expert consultation and a technically demanding image-based tool. That gap is exactly what the proposed system is designed to fill, by allowing farmers to describe what they see in plain terms and receiving a meaningful, actionable response in return.

TABLE II: Comparison of Existing Systems with Proposed System

Feature	Plantix	Leaf Doctor	AgroAI	Proposed System
Primary Input	Image	Image	Image	Symptom-based
Works Without Image	No	No	No	Yes
Disease Identification	Yes	No	Yes	Yes
Treatment Suggestions	Yes	No	Yes	Yes
Confidence Score	No	No	No	Yes
Lightweight Architecture	No	Yes	No	Yes
Rural Accessibility	Low	Low	Low	High
User-Friendly Interface	Moderate	Moderate	Complex	Simple

IV. METHODOLOGY

The development of the proposed system followed a structured five-phase process covering data collection, preprocessing, model development, web application development, and system integration. Each phase involved deliberate technical decisions made with the practical constraints of the project firmly in mind.

4.1 Data Collection

The dataset was built from two primary sources — the PlantVillage dataset hosted by Pennsylvania State University and supplementary plant disease datasets available through Kaggle. Rather than relying on image data, the focus was on constructing a structured symptom dataset by mapping observable plant conditions — leaf discoloration, dark spots, wilting, dryness, curling, and fungal growth — to their corresponding disease labels. This mapping process drew on established agricultural literature and verified disease descriptions from the PlantVillage repository.

The final dataset consisted of 4,460 records covering fifteen plant diseases across five crop categories, providing sufficient diversity to train a robust and generalizable prediction model.

TABLE III: Dataset Composition

Crop	Diseases Covered	Symptom Records
Tomato	4	1,200
Potato	3	950
Pepper	2	680
Corn	3	870
Wheat	3	760
Total	15	4,460

4.2 Data Preprocessing

Raw data collected from multiple sources required thorough cleaning before it could be used for model training. The preprocessing pipeline was implemented in Python using Pandas and NumPy and included the following steps.

Duplicate and inconsistent records were identified and removed. Symptom descriptions using varying terminology for the same observable condition were

standardized to a single canonical form. Categorical variables were encoded using label encoding for ordinal attributes and one-hot encoding for nominal ones. Missing values, present in approximately 3.2% of records, were handled through mode substitution to ensure no records were discarded during cleaning.

For the optional image-based module, uploaded images were resized to 224×224 pixels, pixel values normalized to a range of 0 to 1, and data augmentation techniques including horizontal flipping, rotation, and brightness adjustment applied to improve model generalization.

4.3 Feature Engineering

A total of twenty-two binary symptom features were identified and included in the final feature set. Each input instance is represented as a binary feature vector where each element indicates the presence or absence of a specific observable symptom. The features cover four broad categories — leaf appearance, surface condition, structural changes, and growth abnormalities.

Feature importance analysis was conducted using the Random Forest classifier's built-in Gini impurity ranking to confirm that all twenty-two features contributed meaningfully to prediction accuracy, with none falling below the defined importance threshold of 0.01.

4.4 Model Development

Three classification algorithms were implemented and evaluated.

Naive Bayes was implemented using Scikit-learn's GaussianNB class. As a probabilistic classifier built on Bayes' theorem, it is fast and computationally lightweight, making it a useful baseline for comparison despite its assumption of feature independence.

Logistic Regression was implemented with L2 regularization and the lbfgs solver, configured for multi-class classification. It models class membership as a logistic function of the input features. While interpretable and reasonably accurate, its linear decision boundaries limit effectiveness when symptom profiles overlap significantly between disease classes.

Random Forest was configured with 200 estimators, maximum depth of 20, and Gini impurity criterion. Hyperparameters were selected through grid search cross-validation. By aggregating predictions from multiple decision trees trained on bootstrapped data subsets, Random Forest reduces variance and generalizes well even when symptom patterns are complex and partially overlapping. It was selected as the production model.

based on its superior performance across all evaluation metrics.

All models were trained on 80% of the dataset and evaluated on the remaining 20% using stratified five-fold cross-validation.

4.5 System Workflow

TABLE IV: System Workflow Summary

Step	Process	Technology
1	User selects symptoms via web interface	HTML, CSS, JavaScript
2	Optional crop image upload	Flask, OpenCV
3	Symptom encoding and preprocessing	Python, Scikit-learn
4	Feature vector construction	Pandas, NumPy
5	Disease prediction via Random Forest	Scikit-learn
6	Confidence score generation	Probability estimation
7	Result display with treatment suggestions	Flask, Jinja2
8	Optional prediction record storage	MySQL, Firebase

V. System Architecture

The proposed system follows a three-tier web application architecture built around the principles of modularity, simplicity, and scalability. Each layer has clearly defined responsibilities, ensuring that changes to one component do not disrupt the others.

The **Presentation Layer** is developed using HTML5, CSS3, and JavaScript. It handles all user-facing functionality — rendering the symptom checklist, managing optional image uploads, and displaying prediction results. The interface is intentionally minimal, organized around four symptom groups: Leaf Appearance, Surface Condition, Structural Changes, and Other Indicators. JavaScript manages dynamic

interactions and communicates with the backend through AJAX calls, allowing results to appear without a full page reload.

The **Application Logic Layer** is built on Flask, a lightweight Python web framework. Flask routes incoming requests to the appropriate processing modules, passes inputs through the prediction pipeline, and returns formatted JSON responses to the frontend. The trained Random Forest model and preprocessing pipeline are serialized using joblib and loaded into memory at application startup, keeping prediction latency consistently low without requiring model retraining on each server restart.

The **Data Layer** uses MySQL as the primary relational database, storing disease information, symptom-disease mappings, treatment recommendations, and prediction logs. Firebase handles user authentication and session management, providing a secure and scalable identity solution without the overhead of a custom implementation.

TABLE V: System Architecture Components

Layer	Component	Technology	Responsibility
Presentation	User Interface	HTML5, CSS3, JavaScript	Input collection and result display
Presentation	Dynamic Interactions	JavaScript, AJAX	Async backend communication
Application Logic	Web Framework	Flask (Python)	Request routing and response handling
Application Logic	Image Processing	OpenCV	Image preprocessing and analysis
Machine Learning	Classifier	Scikit-learn (Random Forest)	Disease prediction

Machine Learning	Preprocessing Pipeline	Pandas, NumPy	Feature encoding and transformation
Data	Relational Database	MySQL	Disease data and prediction logs
Data	Cloud Storage	Firebase	User authentication and sync

TABLE VI: API Endpoint Summary

Endpoint	Method	Input	Output
/predict	POST	Symptom feature vector	Predicted disease, confidence score
/upload	POST	Crop image file	Image analysis result
/disease-info	GET	Disease name	Description and treatment
/history	GET	User ID	Past prediction records
/register	POST	User credentials	Authentication token

VI. RESULTS AND PERFORMANCE EVALUATION

6.1 Experimental Setup

All experiments were conducted on an Intel Core i5 system with 8 GB RAM and a 256 GB SSD, running Python 3.10 on Windows 11. The 4,460-record dataset was split into 80% training (3,568 records) and 20% testing (892 records). Stratified five-fold cross-validation was applied during training. Four metrics were used for evaluation — Accuracy, Precision, Recall, and F1-Score.

6.2 Model Performance

The Random Forest classifier outperformed both competing models across all four metrics, confirming that its ensemble approach handles overlapping symptom

patterns more effectively than probabilistic or linear classifiers. Naive Bayes was the fastest but limited by its independence assumption. Logistic Regression performed reasonably well but struggled where symptom profiles overlapped significantly between disease classes.

TABLE VII: Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score	Training Time
Naive Bayes	84.2%	82.7%	81.4%	82.0%	0.8 sec
Logistic Regression	88.6%	87.3%	86.1%	86.7%	2.3 sec
Random Forest	93.4%	92.1%	91.8%	91.9%	6.7 sec

6.3 Disease-Wise Prediction Performance

The Random Forest model maintained strong and consistent performance across all fifteen disease classes, with F1-scores ranging from 89.0% to 94.3%. Diseases with highly distinctive symptom profiles — Corn Smut and Early Blight — achieved the highest scores. Mosaic Virus and Bacterial Wilt recorded comparatively lower scores due to symptom overlap with other disease classes, a challenge inherent to symptom-based classification that is not unique to this system.

TABLE VIII: Disease-Wise Prediction Performance (Random Forest)

Disease	Affected Crop	Precision	Recall	F1-Score
Early Blight	Tomato	94.3%	93.7%	94.0%
Late Blight	Tomato, Potato	92.8%	91.5%	92.1%
Leaf Spot	Tomato, Pepper	91.6%	90.8%	91.2%
Powdery Mildew	Wheat, Pepper	93.1%	92.4%	92.7%
Bacterial Wilt	Tomato, Potato	90.4%	89.6%	90.0%
Fusarium Wilt	Tomato, Corn	91.9%	90.2%	91.0%
Downy Mildew	Wheat	92.5%	91.8%	92.1%
Corn Smut	Corn	94.7%	93.9%	94.3%
Yellow Rust	Wheat	93.8%	92.6%	93.2%
Mosaic Virus	Pepper, Tomato	89.3%	88.7%	89.0%

6.4 System Response Time

TABLE IX: System Response Time Analysis

Input Type	Avg. Response Time	Max Response Time
Symptom input only	1.2 seconds	1.8 seconds
Symptom input with image	2.7 seconds	3.5 seconds
Image input only	2.1 seconds	3.0 seconds
High server load (50 users)	3.4 seconds	4.9 seconds

Symptom-only predictions are returned in an average of 1.2 seconds — well within the threshold for a responsive user experience. Even under a simulated load of fifty concurrent users, maximum response time stayed below 5 seconds, indicating the architecture is sufficiently robust for moderate-scale deployment.

6.5 Comparative Analysis

TABLE X: Comparative Analysis with Existing Systems

Criteria	Plantix	Leaf Doctor	AgroAI	Proposed System
Prediction Accuracy	~90% (image)	N/A	~87%	93.4% (symptom)
Works Without Image	No	No	No	Yes
Response Time	3–5 sec	2–4 sec	4–6 sec	1.2–2.7 sec
Offline Capability	No	No	No	Partial
Treatment Suggestions	Yes	No	Yes	Yes
Confidence Score	No	No	No	Yes
Lightweight	No	Yes	No	Yes
Rural Accessibility	Low	Low	Low	High

The proposed system is the only one among those compared that delivers meaningful disease predictions without requiring image input. Its 93.4% accuracy is competitive with image-based alternatives, and its 1.2-second average response time is faster than all reviewed commercial applications.

VII. Discussion

7.1 Interpretation of Results

The central finding of this research is straightforward — a well-implemented symptom-based prediction system can match the practical accuracy of image-based alternatives while being far more accessible to the users who need it most. Random Forest's 93.4% accuracy is not just a number; it represents a meaningful level of diagnostic reliability that can genuinely inform a farmer's decision about how to respond to a disease threat.

7.2 Practical Implications

The real value of this system is not its accuracy in isolation but its accuracy in context. For a farmer in a rural area with a basic smartphone and inconsistent connectivity, the difference between a system that requires a photograph and one that accepts a description is the difference between getting help and getting none. By designing around symptom input, the proposed system removes the single most significant barrier to adoption in low-resource environments.

7.3 Limitations

The current implementation has limitations that should be stated clearly. Fifteen diseases across five crops is a reasonable starting point but a small fraction of the disease challenges farmers actually face. The model is static — it does not learn from new data after deployment, which means its accuracy may drift over time as disease patterns shift with climate and environmental changes. The system also requires at least a minimal internet connection to function, which excludes users in areas with no connectivity at all. And while the symptom checklist is designed to be intuitive, its usefulness depends on the farmer's ability to accurately identify and describe what they observe — something that cannot always be assumed.

7.4 Future Directions

The most impactful near-term improvement would be expanding the disease and crop coverage through partnerships with agricultural research institutions and by building a crowdsourced symptom reporting mechanism into the application itself. Integrating a lightweight CNN architecture such as MobileNetV3 or EfficientNet-Lite for the image analysis module would meaningfully improve prediction confidence when image input is available. A dedicated Android application with on-device model deployment would extend the system's reach to users without reliable browser access and enable at least partial offline functionality.

VIII. CONCLUSION

This paper presented the design and evaluation of a symptom-based Plant Disease Prediction System built to address a gap that the existing literature has acknowledged but largely left unresolved — the inaccessibility of AI-based disease detection tools for rural and smallholder farmers who lack the equipment, connectivity, and technical literacy that most current systems require.

The system uses a Random Forest classifier trained on 4,460 structured symptom records covering fifteen diseases across five crop categories. Among the three algorithms evaluated, Random Forest achieved the strongest results — 93.4% accuracy, 92.1% precision, 91.8% recall, and 91.9% F1-score — delivered through a lightweight Flask web application in an average of 1.2 seconds. Comparative analysis against Plantix, Leaf Doctor, and AgroAI confirmed that the proposed system is the only solution among those reviewed that provides meaningful disease prediction without requiring image input, while also offering confidence scores and treatment recommendations through an interface designed for non-technical users.

The work has limitations — disease coverage is narrow, the model is static, and full offline capability has not yet been implemented. These are not minor issues and they define the roadmap for future development. Expanding the disease database, integrating a lightweight image analysis module, building an Android application, and adding multilingual support are the priorities that will determine how much practical impact this system ultimately has.

What this research demonstrates, at its core, is that accessible and practical AI tools do not have to sacrifice meaningful accuracy to serve the users who need them most. A system that works reliably in a rural field with minimal prerequisites is more valuable than one that achieves laboratory-level accuracy and sits unused. That is the principle this work was built on, and it is a principle worth carrying forward into future research on AI for agriculture.²

IX. REFERENCES

- [1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, Sep. 2016.
- [2] E. C. Too, L. Yujian, S. Njuki, and L. Yingchun, "A comparative study of fine-tuning deep learning models for plant disease identification," *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, Jun. 2019.
- [3] K. G. Liakos, P. Busato, D. Moshou, S. Pearson, and D. Bochtis, "Machine learning in agriculture: A review," *Sensors*, vol. 18, no. 8, p. 2674, Aug. 2018.
- [4] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, Feb. 2018.
- [5] R. Sharma, H. Patel, and D. Verma, "Symptom-based plant disease prediction using machine learning techniques," *International Journal of Agricultural Technology*, vol. 17, no. 3, pp. 1045–1058, May 2021.
- [6] R. Sharma, M. Singh, and A. Gupta, "Hybrid CNN model for plant disease classification using leaf image analysis," *Journal of Artificial Intelligence in Agriculture*, vol. 6, no. 2, pp. 88–97, Mar. 2023.
- [7] L. Chen, Y. Wang, and H. Zhao, "Explainable AI for plant disease detection systems," *IEEE Access*, vol. 12, pp. 34521–34535, Jan. 2024.
- [8] R. Gupta and M. Jain, "Lightweight AI models for mobile plant disease detection," *International Journal of Smart Agriculture Systems*, vol. 3, no. 1, pp. 22–34, Feb. 2025.
- [9] S. Ramesh and D. Vydeki, "Recognition and classification of paddy leaf diseases using optimized deep neural network," *International Journal of Intelligent Engineering and Systems*, vol. 13, no. 1, pp. 1–10, Feb. 2020.
- [10] S. Zhang, W. Huang, and C. Zhang, "Three-channel convolutional neural networks for vegetable leaf disease recognition," *Cognitive Systems Research*, vol. 53, pp. 31–41, Jan. 2019.
- [11] P. Kumar and A. Singh, "Symptom-based plant disease prediction using machine learning techniques," *International Journal of Agricultural Technology*, vol. 17, no. 4, pp. 1123–1135, Jul. 2021.
- [12] Food and Agriculture Organization of the United Nations (FAO), "Digital technologies in agriculture and rural areas: Status report," FAO, Rome, Italy, 2019. [Online]. Available: <https://www.fao.org>
- [13] F. Chollet, *Deep Learning with Python*. Shelter Island, NY, USA: Manning Publications, 2018.
- [14] Scikit-learn Developers, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://scikit-learn.org>
- [15] PlantVillage Project, Pennsylvania State University, "PlantVillage disease dataset," 2015. [Online]. Available: <https://plantvillage.psu.edu>
- [16] Kaggle Inc., "Plant disease datasets repository," 2023. [Online]. Available: <https://www.kaggle.com/datasets>